

```

#!/usr/bin/env bash
# =====
# deploy-cloud-mcp-suite.sh
# Master deployment script – Nexus VPS (Ubuntu 24.04, rootless Podman)
#
# Deploys:
#   pCloud MCP          → port 8201 → pcloud-mcp.shannonjlove.cloud
#   Backblaze B2 MCP    → port 8202 → b2-mcp.shannonjlove.cloud
#   MediaFire MCP       → port 8203 → mediafire-mcp.shannonjlove.cloud
#   MEGA MCP            → port 8204 → mega-mcp.shannonjlove.cloud
#   Google Drive MCP    → port 8205 → gdrive-mcp.shannonjlove.cloud
#   rclone iDrive E2    → configured on host (no container)
#
# Run as: shannon (rootless Podman user)
# =====

set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; NC='\033[0m'

info() { echo -e "${CYAN}[INFO]${NC}  $*"; }
success() { echo -e "${GREEN}[ OK ]${NC}  $*"; }
warn() { echo -e "${YELLOW}[WARN]${NC}  $*"; }
step() { echo -e "\n${BOLD}—— $* ——${NC}"; }
fail() { echo -e "${RED}[FAIL]${NC}  $*"; exit 1; }

SUITE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
DEPLOY_BASE="/opt/mcp-servers"
QUADLET_DIR="${HOME}/.config/containers/systemd"

# — Verify environment —————
step "Pre-flight checks"
command -v podman &>/dev/null || fail "podman not found"
command -v systemctl &>/dev/null || fail "systemctl not found"
[[ "$(id -u)" -ne 0 ]] || fail "Run as non-root (rootless Podman user)"
success "Running as $(whoami) – rootless Podman confirmed"

# — Port map —————
declare -A SERVERS=(
    [pcloud]="8201"
    [backblaze]="8202"
    [mediafire]="8203"
    [mega]="8204"

```

```

        [gdrive]="8205"
    )
    declare -A CONTAINERS=(
        [pcloud]="pcloud-mcp"
        [backblaze]="backblaze-mcp"
        [mediafire]="mediafire-mcp"
        [mega]="mega-mcp"
        [gdrive]="gdrive-mcp"
    )
    declare -A SUBDOMAINS=(
        [pcloud]="pcloud-mcp.shannonjlove.cloud"
        [backblaze]="b2-mcp.shannonjlove.cloud"
        [mediafire]="mediafire-mcp.shannonjlove.cloud"
        [mega]="mega-mcp.shannonjlove.cloud"
        [gdrive]="gdrive-mcp.shannonjlove.cloud"
    )

# — Step 1: Create server directories and copy files —————
step "Step 1: Staging server files to ${DEPLOY_BASE}"
sudo mkdir -p "${DEPLOY_BASE}"
sudo chown -R "$(whoami):$(whoami)" "${DEPLOY_BASE}"

for SRV in "${!SERVERS[@]}"; do
    SRV_DIR="${DEPLOY_BASE}/${SRV}"
    mkdir -p "${SRV_DIR}"
    cp -v "${SUITE_DIR}/${SRV}/*.py" "${SRV_DIR}/" 2>/dev/null || true
    cp -v "${SUITE_DIR}/${SRV}/requirements.txt" "${SRV_DIR}/" 2>/dev/null || true
    success "${SRV} → ${SRV_DIR}"
done

# Create credentials dir for gdrive
mkdir -p "${DEPLOY_BASE}/gdrive/credentials"

# — Step 2: Create env files (if they don't already exist) —————
step "Step 2: Creating env file templates (fill in credentials)"

# pCloud
ENV="${DEPLOY_BASE}/pcloud/pcloud.env"
if [[ ! -f "${ENV}" ]]; then
    cat > "${ENV}" <<'EOF'
# pCloud credentials
PCLOUD_ACCESS_TOKEN=YOUR_PCLOUD_OAUTH2_TOKEN_HERE
PCLOUD_REGION=us
# Set to 'eu' if your account is on EU servers (eapi.pcloud.com)
EOF
    warn "Created ${ENV} – add your pCloud OAuth2 token"
else

```

```

        info "${ENV} already exists - skipping"
    fi

    # Backblaze
    ENV="${DEPLOY_BASE}/backblaze/backblaze.env"
    if [[ ! -f "${ENV}" ]]; then
        cat > "${ENV}" <<'EOF'
    # Backblaze B2 credentials
    B2_APPLICATION_KEY_ID=YOUR_B2_KEY_ID_HERE
    B2_APPLICATION_KEY=YOUR_B2_APP_KEY_HERE
    EOF

        warn "Created ${ENV} - add your B2 application key"
    else
        info "${ENV} already exists - skipping"
    fi

    # MediaFire
    ENV="${DEPLOY_BASE}/mediafire/mediafire.env"
    if [[ ! -f "${ENV}" ]]; then
        cat > "${ENV}" <<'EOF'
    # MediaFire credentials (developer portal: https://www.mediafire.com/developer/)
    MEDIAFIRE_APP_ID=YOUR_APP_ID_HERE
    MEDIAFIRE_API_KEY=YOUR_API_KEY_HERE
    MEDIAFIRE_EMAIL=YOUR_MEDIAFIRE_EMAIL
    MEDIAFIRE_PASSWORD=YOUR_MEDIAFIRE_PASSWORD
    EOF

        warn "Created ${ENV} - add your MediaFire credentials"
    else
        info "${ENV} already exists - skipping"
    fi

    # MEGA
    ENV="${DEPLOY_BASE}/mega/mega.env"
    if [[ ! -f "${ENV}" ]]; then
        cat > "${ENV}" <<'EOF'
    # MEGA credentials
    MEGA_EMAIL=YOUR_MEGA_EMAIL
    MEGA_PASSWORD=YOUR_MEGA_PASSWORD
    EOF

        warn "Created ${ENV} - add your MEGA credentials"
    else
        info "${ENV} already exists - skipping"
    fi

    # Google Drive
    ENV="${DEPLOY_BASE}/gdrive/gdrive.env"
    if [[ ! -f "${ENV}" ]]; then

```

```

    cat > "${ENV}" <<'EOF'
# Google Drive credentials — choose ONE auth method:

# Option A: Service Account (recommended for server deployments)
GDRIVE_SERVICE_ACCOUNT_JSON=/credentials/service-account.json
# GDRIVE_IMPERSONATE_EMAIL=user@yourdomain.com # for domain-wide delegation

# Option B: OAuth2 user credentials (comment out Option A)
# GDRIVE_CREDENTIALS_JSON=/credentials/credentials.json
# GDRIVE_TOKEN_JSON=/credentials/token.json
EOF
    warn "Created ${ENV} — add your Google Drive credentials"
    warn "Place your service-account.json in ${DEPLOY_BASE}/gdrive/credentials/"
else
    info "${ENV} already exists — skipping"
fi

# — Step 3: Secure env files —————
step "Step 3: Securing env files (chmod 600)"
for SRV in "${!SERVERS[@]}"; do
    ENV_FILE="${DEPLOY_BASE}/${SRV}/${SRV}.env"
    [[ -f "${ENV_FILE}" ]] && chmod 600 "${ENV_FILE}" && success "Secured ${ENV_F
done

# — Step 4: Install Podman quadlet files —————
step "Step 4: Installing Podman quadlet files"
mkdir -p "${QUADLET_DIR}"

for SRV in "${!SERVERS[@]}"; do
    QUAD_SRC="${SUITE_DIR}/${SRV}/${CONTAINERS[$SRV]}.container"
    QUAD_DST="${QUADLET_DIR}/${CONTAINERS[$SRV]}.container"
    if [[ -f "${QUAD_SRC}" ]]; then
        # Patch Deploy_Base path into quadlet if needed
        sed "s|/opt/mcp-servers|${DEPLOY_BASE}|g" "${QUAD_SRC}" > "${QUAD_DST}"
        success "Installed quadlet: ${QUAD_DST}"
    else
        warn "Quadlet not found: ${QUAD_SRC}"
    fi
done

# — Step 5: Reload systemd and start services —————
step "Step 5: Reloading systemd and starting services"
systemctl --user daemon-reload
success "systemd reloaded"

STARTED=()
FAILED=()

```

```

for SRV in "${!SERVERS[@]}"; do
    SVC="${CONTAINERS[$SRV]}"
    info "Starting ${SVC}..."
    if systemctl --user enable --now "${SVC}" 2>&1; then
        success "${SVC} started"
        STARTED+=("${SVC}")
    else
        warn "${SVC} failed to start - check: journalctl --user -u ${SVC} -n 50"
        FAILED+=("${SVC}")
    fi
done

```

```

# — Step 6: Verify ports —————
step "Step 6: Verifying local ports"
sleep 5 # give containers time to bind

```

```

for SRV in "${!SERVERS[@]}"; do
    PORT="${SERVERS[$SRV]}"
    if ss -tlnp 2>/dev/null | grep -q ":${PORT} "; then
        success "Port ${PORT} → ${CONTAINERS[$SRV]} ✓"
    else
        warn "Port ${PORT} not listening yet for ${CONTAINERS[$SRV]}"
    fi
done

```

```

# — Step 7: NPM proxy instructions —————
step "Step 7: Nginx Proxy Manager — proxy host setup"
echo ""
echo -e "${BOLD}Add these proxy hosts in NPM (http://admin.shannonjlove.cloud → P
echo ""
printf "  %-40s  %-18s  %s\n" "DOMAIN" "FORWARD HOST:PORT" "SSL"
printf "  %-40s  %-18s  %s\n" "-----" "-----" "----"
for SRV in "${!SERVERS[@]}"; do
    PORT="${SERVERS[$SRV]}"
    DOM="${SUBDOMAINS[$SRV]}"
    printf "  %-40s  127.0.0.1:%-8s  Let's Encrypt\n" "${DOM}" "${PORT}"
done

```

```

echo ""
echo -e "${BOLD}NPM proxy host settings for each entry:${NC}"
echo "  Scheme:          http"
echo "  Forward Hostname: 127.0.0.1"
echo "  Forward Port:     (see above)"
echo "  Cache Assets:     OFF"
echo "  Block Exploits:   ON"
echo "  Websockets:       ON"

```

```
echo "    SSL:                Request new cert → Force SSL → HSTS"
```

```
echo ""
```

```
# — Step 8: Claude Desktop MCP config snippet —————
```

```
step "Step 8: Claude Desktop config snippet"
```

```
CLAUDE_CONFIG="${HOME}/.config/claude-desktop/mcp_servers.json"
```

```
cat << 'JSONEOF'
```

```
Add to your Claude Desktop MCP config (~/.config/claude-desktop/mcp_servers.json)
```

```
{
  "mcpServers": {
    "pcloud": {
      "url": "https://pcloud-mcp.shannonjlove.cloud/mcp",
      "transport": "streamable_http"
    },
    "backblaze": {
      "url": "https://b2-mcp.shannonjlove.cloud/mcp",
      "transport": "streamable_http"
    },
    "mediafire": {
      "url": "https://mediafire-mcp.shannonjlove.cloud/mcp",
      "transport": "streamable_http"
    },
    "mega": {
      "url": "https://mega-mcp.shannonjlove.cloud/mcp",
      "transport": "streamable_http"
    },
    "gdrive-local": {
      "url": "https://gdrive-mcp.shannonjlove.cloud/mcp",
      "transport": "streamable_http"
    }
  }
}
JSONEOF
```

```
# — Step 9: Deploy rclone for iDrive E2 —————
```

```
step "Step 9: rclone iDrive E2 deployment"
```

```
RCLONE_SCRIPT="${SUITE_DIR}/rclone-idrivee2/deploy-rclone-idrivee2.sh"
```

```
if [[ -f "${RCLONE_SCRIPT}" ]]; then
```

```
  chmod +x "${RCLONE_SCRIPT}"
```

```
  echo ""
```

```
  info "rclone deployment script ready. Run separately with your credentials:"
```

```
  info "  bash ${RCLONE_SCRIPT}"
```

```
  info ""
```

```
  info "Or set env vars first for non-interactive run:"
```

```
  info "  export IDRIVEE2_ACCESS_KEY=YOUR_KEY"
```

```

info " export IDRIVEE2_SECRET_KEY=YOUR_SECRET"
info " export IDRIVEE2_ENDPOINT=https://YOUR_ACCOUNT.idrivee2-N.com"
info " bash ${RCLONE_SCRIPT}"
fi

# --- Summary ---
echo ""
echo -e "${BOLD} "
echo -e "${BOLD} Cloud MCP Suite - Deployment Summary "
echo -e "${BOLD} "
echo ""
echo -e "${BOLD} Started (${#STARTED[@]}):${NC}"
for s in "${STARTED[@]"; do echo "  ${s}"; done
echo ""
if [[ ${#FAILED[@]} -gt 0 ]]; then
    echo -e "${BOLD} Failed (${#FAILED[@]}):${NC}"
    for s in "${FAILED[@]"; do echo "  ${s}"; done
fi
echo ""
echo -e "${BOLD} Fill in credentials:${NC}"
for SRV in "${!SERVERS[@]"; do
    echo " vim ${DEPLOY_BASE}/${SRV}/${SRV}.env"
done
echo ""
echo -e "${BOLD} Restart after editing credentials:${NC}"
for SRV in "${!SERVERS[@]"; do
    echo " systemctl --user restart ${CONTAINERS[$SRV]}"
done
echo ""
echo -e "${BOLD} Monitor logs:${NC}"
for SRV in "${!SERVERS[@]"; do
    echo " journalctl --user -u ${CONTAINERS[$SRV]} -f"
done
echo ""

```